CONTRACT WOLF

*Security Assessment*

# Lobsterswap DEX

Verified on November 2, 2026

## SUMMARY

| Project | CHAIN | METHODOLOGY |
|---|---|---|
| Lobsterswap DEX | OZO | Manual & Automatic Analysis |

| FILES | DELIVERY | TYPE |
|---|---|---|
| Live Contracts | November 2, 2026 | Standard Audit |



| 10 | 0 | 1 | 1 | 3 | 5 | 0 |
|---|---|---|---|---|---|---|
| Total Findings | Critical | Major | Medium | Minor | Informational | Resolved |

| | |
|---|---|
| 0 Critical | An exposure that can affect the contract functions in several events that can risk and disrupt the contract |
| 1 Major | An opening & exposure to manipulate the contract in an unwanted manner |
| 1 Medium | An opening that could affect the outcome in executing the contract in a specific situation |
| 3 Minor | An opening but doesn't have an impact on the functionality of the contract |
| 5 Informational | An opening that consists information but will not risk or affect the contract |
| 0 Resolved | ContractWolf's findings has been acknowledged & resolved by the project |

**STATUS** ⚠ **REVIEW REQUIRED**

# TABLE OF CONTENTS | Lobsterswap DEX

## Summary

Project Summary
Findings Summary
Disclaimer
Scope of Work
Auditing Approach

## Project Information

Token/Project Details
Inheritance Graph
Call Graph

## Findings

Issues
SWC Attacks
CW Assessment
Fixes & Recommendation
Audit Comments

# DISCLAIMER | Lobsterswap DEX

**ContractWolf** audits and reports should not be considered as a form of project's "Advertisement" and does not cover any interaction and assessment from "Project Contract" to "External Contracts" such as PancakeSwap, UniSwap, SushiSwap or similar.

**ContractWolf** does not provide any warranty on its released report and should not be used as a decision to invest into audited projects.

**ContractWolf** provides a transparent report to all its "Clients" and to its "Clients Participants" and will not claim any guarantee of bug-free code within its **SMART CONTRACT**.

**ContractWolf**'s presence is to analyze, audit and assess the Client's Smart Contract to find any underlying risk and to eliminate any logic and flow errors within its code.

*Each company or project should be liable to its security flaws and functionalities.*

# SCOPE OF WORK | Lobsterswap DEX

**Lobsterswap DEX** team has agreed and provided us with the files that need to be tested (*Github, BSCscan, Etherscan, Local files etc*). The scope of audit is the main contract.

The goal of this engagement is to identify if there is a possibility of security flaws in the implementation of smart contract and its systems.

ContractWolf will be focusing on contract issues and functionalities along with the project claims from smart contract to their website, whitepaper, repository which has been provided by **Lobsterswap DEX**.

# AUDITING APPROACH | Lobsterswap DEX

Every line of code along with its functionalities will undergo manual review to check for security issues, quality of logic and contract scope of inheritance. The manual review will be done by our team that will document any issues that they discovered.

## METHODOLOGY

The auditing process follows a routine series of steps :

1. Code review that includes the following :
   - Review of the specifications, sources and instructions provided to ContractWolf to make sure we understand the size, scope and functionality of the smart contract.
   - Manual review of code. Our team will have a process of reading the code line-by-line with the intention of identifying potential vulnerabilities, underlying and hidden security flaws.

2. Testing and automated analysis that includes :
   - Testing the smart contract function with common test cases and scenarios to ensure that it returns the expected results.

3. Best practices and ethical review. The team will review the contract with the aim to improve efficiency, effectiveness, clarifications, maintainability, security and control within the smart contract.

4. Recommendations to help the project take steps to eliminate or minimize threats and secure the smart contract.

# TOKEN DETAILS | Lobsterswap DEX

LOBSTER SWAP

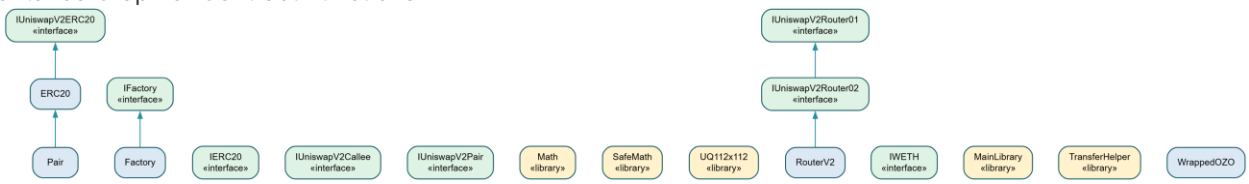
Lobster Swap is a delightful underwater adventure! Prepare to be shell-shocked by our powerful DEX, offering seamless trading within the Ozone Chain ecosystem.

# SOURCE

| | |
|---|---|
| Source | *Live Contracts*<br>*Factory 0x19A3F979A5c6d42359F6dCDE00b5bd0544a3b191*<br>*Router 0xa7B539333E7a910EAF3857DfFE546a2Da7287823*<br>*Wrapped OZO 0x52b238A907AB16afEc79cc0580E5B2e0E199D7f9* |

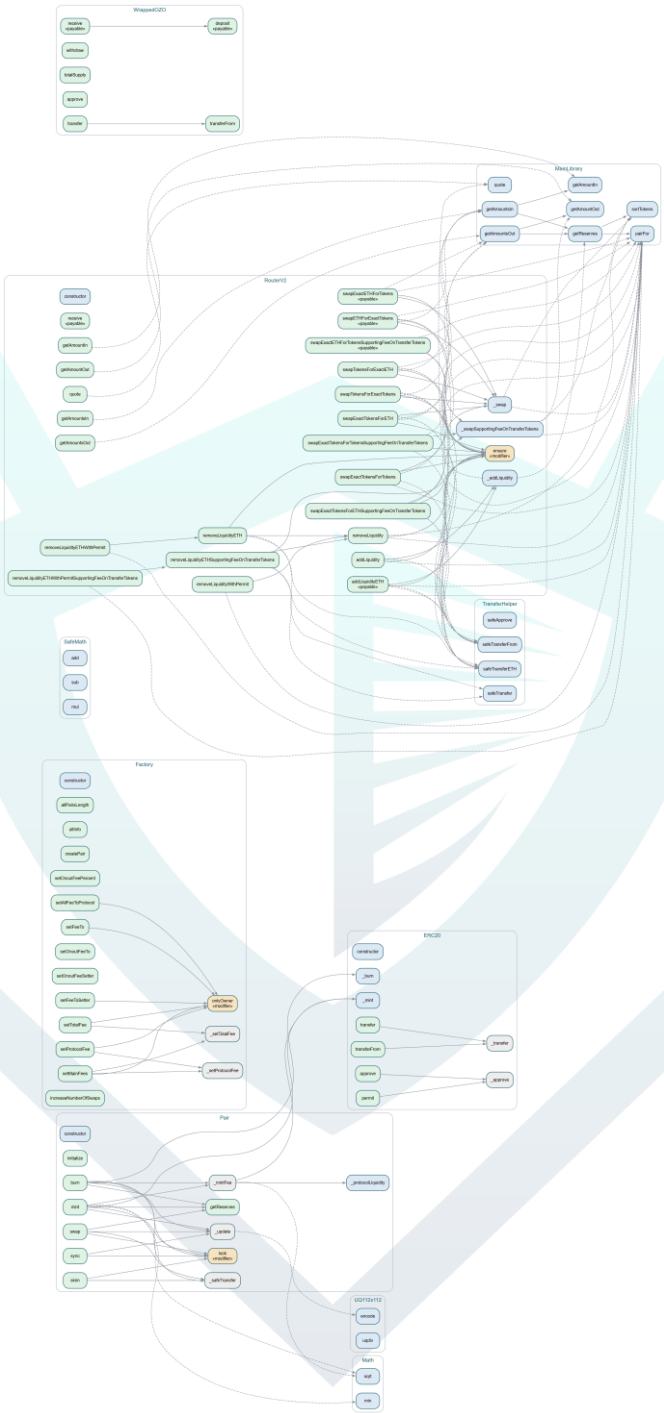
# INHERITANCE GRAPH | Lobsterswap DEX

Inheritance Graph of Contract Functions



# CALL GRAPH | Lobsterswap DEX

Call Graph of Contract Functions



# FINDINGS | Lobsterswap DEX



| 10 | 0 | 1 | 1 | 3 | 5 | 0 |
|---|---|---|---|---|---|---|
| Total Findings | Critical | Major | Medium | Minor | Informational | Resolved |

This report has been prepared to state the issues and vulnerabilities for Lobsterswap DEX through this audit. The goal of this report findings is to identify specifically and fix any underlying issues and errors

| ID | Title | File & Line # | Severity | Status |
|---|---|---|---|---|
| FEE-CONTROL | Admin can raise the swap fee to 99.9% and route all of it to itself | Factory.sol, L: 108 | Major | Pending |
| PROTOCOL-FEE-RETRO | Protocol fee changes apply retroactively to fees LPs already earned | Pair.sol, L: 90 | Medium | Pending |
| SWC-134 | WOZO withdraw forwards only 2300 gas | WrappedOZO.sol, L: 37 | Minor | Pending |
| SWC-101 | Checked arithmetic on overflow-intended TWAP math can brick pairs | Pair.sol, L: 78 | Minor | Pending |
| CW-002 | Single-step admin transfer with no zero-address check | Factory.sol, L: 90; Factory.sol, L: 99 | Minor | Pending |
| CW-005 | Onout fee share divides before it multiplies | Pair.sol, L: 110 | Informational | Pending |
| SWC-102 | Factory and Router compiled with outdated Solidity 0.8.0 | Factory.sol, L: 2; RouterV2.sol, L: 2 | Informational | Pending |
| SWC-103 | Floating pragma across all contracts | Factory.sol, L: 2; Pair.sol, L: 2; RouterV2.sol, L: 2; WrappedOZO.sol, L: 2 | Informational | Pending |
| ROUTER-ABI | Router quote helpers break the standard Uniswap V2 interface | RouterV2.sol, L: 408 | Informational | Pending |
| FACTORY-CALL-GAS | Every swap and liquidity event makes repeated factory calls | Pair.sol, L: 216 | Informational | Pending |

# SWC ATTACKS | Lobsterswap DEX

Smart Contract Weakness Classification and Test Cases

| ID | Description | Status |
|---|---|---|
| SWC-100 | Function Default Visibility | Passed |
| SWC-101 | Integer Overflow and Underflow | Not Passed |
| SWC-102 | Outdated Compiler Version | Not Passed |
| SWC-103 | Floating Pragma | Not Passed |
| SWC-104 | Unchecked Call Return Value | Passed |
| SWC-105 | Unprotected Ether Withdrawal | Passed |
| SWC-106 | Unprotected SELF DESTRUCT Instruction | Passed |
| SWC-107 | Reentrancy | Passed |
| SWC-108 | State Variable Default Visibility | Passed |
| SWC-109 | Uninitialized Storage Pointer | Passed |
| SWC-110 | Assert Violation | Passed |
| SWC-111 | Use of Deprecated Solidity Functions | Passed |
| SWC-112 | Delegate call to Untrusted Callee | Passed |
| SWC-113 | DoS with Failed Call | Passed |
| SWC-114 | Transaction Order Dependence | Passed |
| SWC-115 | Authorization through tx.origin | Passed |
| SWC-116 | Block values as a proxy for time | Passed |
| SWC-117 | Signature Malleability | Passed |
| SWC-118 | Incorrect Constructor Name | Passed |
| SWC-119 | Shadowing State Variables | Passed |
| SWC-120 | Weak Sources of Randomness from Chain Attributes | Passed |
| SWC-121 | Missing Protection against Signature Replay Attacks | Passed |
| SWC-122 | Lack of Proper Signature Verification | Passed |

| ID | Description | Status |
|---|---|---|
| SWC-123 | Requirement Violation | Passed |
| SWC-124 | Write to Arbitrary Storage Location | Passed |
| SWC-125 | Incorrect Inheritance Order | Passed |
| SWC-126 | Insufficient Gas Griefing | Passed |
| SWC-127 | Arbitrary Jump with Function Type Variable | Passed |
| SWC-128 | DoS With Block Gas Limit | Passed |
| SWC-129 | Typographical Error | Passed |
| SWC-130 | Right-To-Left-Override control character(U+202E) | Passed |
| SWC-131 | Presence of unused variables | Passed |
| SWC-132 | Unexpected Ether balance | Passed |
| SWC-133 | Hash Collisions With Multiple Variable Arguments | Passed |
| SWC-134 | Message call with hardcoded gas amount | Not Passed |
| SWC-135 | Code With No Effects | Passed |
| SWC-136 | Unencrypted Private Data On-Chain | Passed |

# CW ASSESSMENT | Lobsterswap DEX

ContractWolf Vulnerability and Security Tests

| ID | Name | Description | Status |
|---|---|---|---|
| CW-001 | Multiple Version | Presence of multiple compiler version across all contracts | ✗ |
| CW-002 | Incorrect Access Control | Additional checks for critical logic and flow | ✗ |
| CW-003 | Payable Contract | A function to withdraw ether should exist otherwise the ether will be trapped | ✓ |
| CW-004 | Custom Modifier | major recheck for custom modifier logic | ✓ |
| CW-005 | Divide Before Multiply | Performing multiplication before division is generally better to avoid loss of precision | ✗ |
| CW-006 | Multiple Calls | Functions with multiple internal calls | ✗ |
| CW-007 | Deprecated Keywords | Use of deprecated functions/operators such as block.blockhash() for blockhash(), msg.gas for gasleft(), throw for revert(), sha3() for keccak256(), callcode() for delegatecall(), suicide() for selfdestruct(), constant for view or var for actual type name should be avoided to prevent unintended errors with newer compiler versions | ✓ |
| CW-008 | Unused Contract | Presence of an unused, unimported or uncalled contract | ✓ |
| CW-009 | Assembly Usage | Use of EVM assembly is error-prone and should be avoided or double-checked for correctness | ✗ |
| CW-010 | Similar Variable Names | Variables with similar names could be confused for each other and therefore should be avoided | ✓ |
| CW-011 | Commented Code | Removal of commented/unused code lines | ✓ |
| CW-012 | SafeMath Override | SafeMath is no longer needed starting with Solidity v0.8+. The compiler now has built-in overflow checking. | ✗ |

## FIXES & RECOMMENDATION

### FEE-CONTROL | Admin can raise the swap fee to 99.9% and route all of it to itself

`feeToSetter`, a single EOA on chain, can call `setMainFees()` or `setTotalFee()` at any time with no delay. `_setTotalFee()` accepts any value up to `MAX_TOTAL_FEE_PERCENT - 1`, which is 999 out of 1000, a 99.9% fee on every swap. With `protocolFee` at `10000` and `allFeeToProtocol` set, which is the live configuration, all fee growth is minted as LP tokens to `feeTo`, so a raised fee is captured by the admin, bounded only by each trader's slippage setting.

```
uint public override constant MAX_TOTAL_FEE_PERCENT = 1_000;

function setMainFees(uint _totalFee, uint _protocolFee) external override onlyOwner {
    _setTotalFee(_totalFee);
    _setProtocolFee(_protocolFee);
    require(totalFee == _totalFee && protocolFee == _protocolFee, 'Factory:
CANNOT_CHANGE');
}

function _setTotalFee(uint _totalFee) private {
    require(_totalFee >= 0 && _totalFee <= MAX_TOTAL_FEE_PERCENT - 1, 'Factory:
FORBIDDEN_FEE');
    totalFee = _totalFee;
}
```

#### Recommendation

Cap the swap fee at a sane hard limit such as 1% (`10` out of `1000`), and move `feeToSetter` behind a multisig with a timelock so traders and LPs can react before a new fee takes effect.

```
uint public constant MAX_SWAP_FEE = 10; // 1% of MAX_TOTAL_FEE_PERCENT

function _setTotalFee(uint _totalFee) private {
    require(_totalFee <= MAX_SWAP_FEE, 'Factory: FORBIDDEN_FEE');
    totalFee = _totalFee;
}

// deploy with feeToSetter = TimelockController(minDelay >= 48 hours) owned by a multisig
```

## PROTOCOL-FEE-RETRO | Protocol fee changes apply retroactively to fees LPs already earned

`_mintFee()` only runs on `mint()` and `burn()`, and it reads `protocolFee` and `allFeeToProtocol` from the factory at that moment. The share is then applied to all fee growth since `kLast`, however long ago that was. The admin can raise the protocol share or set `allFeeToProtocol` just before a liquidity event and take fees that accrued while LPs were promised a larger share.

```
function _mintFee(uint112 _reserve0, uint112 _reserve1) private returns (bool feeOn) {
    address feeTo = IFactory(factory).feeTo();
    ...
    uint protocolFee = IFactory(factory).protocolFee();
    ...
            uint liquidity = _protocolLiquidity(rootK, rootKLast);

function _protocolLiquidity(uint rootK, uint rootKLast) internal view returns(uint
liquidity) {
    bool allFeeToProtocol = IFactory(factory).allFeeToProtocol();
    ...
    uint denominator = rootK.mul(allFeeToProtocol ? 0 : feeMultiplier).add(rootKLast);
    liquidity = numerator / denominator;
}
```

**Recommendation**

Settle accrued fees under the old terms before any change takes effect: expose a permissionless checkpoint on each pair and timelock the factory setters, so keepers or LPs can checkpoint every pair during the delay.

```
// Pair.sol
function checkpointFee() external lock {
    (uint112 _reserve0, uint112 _reserve1,) = getReserves();
    bool feeOn = _mintFee(_reserve0, _reserve1);
    if (feeOn) kLast = uint(_reserve0) * _reserve1;
}

// Factory.sol: queue fee changes, apply after FEE_DELAY
function setProtocolFee(uint _protocolFee) external override onlyOwner {
    pendingProtocolFee = _protocolFee;
    protocolFeeEta = block.timestamp + FEE_DELAY;
}
```

## SWC-134 | WOZO withdraw forwards only 2300 gas

`withdraw()` pays out with `payable(msg.sender).transfer(wad)`, which forwards a fixed 2300 gas stipend. A multisig, a proxy wallet or any contract whose `receive()` does more than emit an event cannot unwrap its WOZO and must first move it to an EOA. A future gas repricing of SLOAD or CALL on Ozone can break unwrapping for contracts that work today.

```
function withdraw(uint256 wad) public {
    require(balanceOf[msg.sender] >= wad, "Insufficient balance");
    balanceOf[msg.sender] -= wad;
    payable(msg.sender).transfer(wad);
    emit Withdrawal(msg.sender, wad);
}
```

**Recommendation**

Send OZO with a low-level `call` and check the result. The balance is already reduced before the call, so the checks-effects-interactions order keeps it safe against reentrancy.

```
function withdraw(uint256 wad) public {
    require(balanceOf[msg.sender] >= wad, "Insufficient balance");
    balanceOf[msg.sender] -= wad;
    (bool ok, ) = payable(msg.sender).call{value: wad}("");
    require(ok, "OZO transfer failed");
    emit Withdrawal(msg.sender, wad);
}
```

## SWC-101 | Checked arithmetic on overflow-intended TWAP math can brick pairs

The Uniswap V2 oracle code was ported to Solidity `0.8` without unchecked. `blockTimestamp - blockTimestampLast` underflows and reverts once `block.timestamp % 2**32` wraps in February 2106, and the `price0CumulativeLast` and `price1CumulativeLast` sums revert instead of wrapping. Either revert sits inside `_update()`, so `mint()`, `burn()`, `swap()` and `sync()` all fail and the pair is frozen.

```
uint32 blockTimestamp = uint32(block.timestamp % 2**32);
uint32 timeElapsed = blockTimestamp - blockTimestampLast; // overflow is desired
if (timeElapsed > 0 && _reserve0 != 0 && _reserve1 != 0) {
    // * never overflows, and + overflow is desired
    price0CumulativeLast += uint(UQ112x112.encode(_reserve1).uqdiv(_reserve0)) *
timeElapsed;
    price1CumulativeLast += uint(UQ112x112.encode(_reserve0).uqdiv(_reserve1)) *
timeElapsed;
}
```

**Recommendation**

Wrap the timestamp delta and both accumulator updates in an unchecked block, restoring the modular arithmetic the Uniswap V2 oracle design relies on.

```
uint32 blockTimestamp = uint32(block.timestamp % 2**32);
unchecked {
    uint32 timeElapsed = blockTimestamp - blockTimestampLast;
    if (timeElapsed > 0 && _reserve0 != 0 && _reserve1 != 0) {
        price0CumulativeLast += uint(UQ112x112.encode(_reserve1).uqdiv(_reserve0)) *
timeElapsed;
        price1CumulativeLast += uint(UQ112x112.encode(_reserve0).uqdiv(_reserve1)) *
timeElapsed;
    }
}
```

## CW-002 | Single-step admin transfer with no zero-address check

`setFeeToSetter()` and `setOnoutFeeSetter()` overwrite the admin in one call and accept `address(0)`. A typo or a wrong paste hands the role to an address nobody controls, permanently, with no way to recover it. Every fee setting, `feeTo` and the Onout split would then be frozen at their current values.

```
function setFeeToSetter(address _feeToSetter) external override onlyOwner {
    feeToSetter = _feeToSetter;
}

function setOnoutFeeSetter(address _OnoutFeeToSetter) external override {
    require(msg.sender == OnoutFeeSetter, 'Factory: FORBIDDEN');
    OnoutFeeSetter = _OnoutFeeToSetter;
}
```

**Recommendation**

Use a two-step handover where the new admin must accept the role, and reject `address(0)`. Apply the same pattern to `OnoutFeeSetter`.

```
address public pendingFeeToSetter;

function setFeeToSetter(address _feeToSetter) external override onlyOwner {
    require(_feeToSetter != address(0), 'Factory: ZERO_ADDRESS');
    pendingFeeToSetter = _feeToSetter;
}

function acceptFeeToSetter() external {
    require(msg.sender == pendingFeeToSetter, 'Factory: FORBIDDEN');
    feeToSetter = pendingFeeToSetter;
    pendingFeeToSetter = address(0);
}
```

## CW-005 | Onout fee share divides before it multiplies

`_mintFee()` takes `liquidity / 100` first and then multiplies by `OnoutFeePercent`, so up to 99 units of LP token per fee mint are truncated from the Onout share and fall to `feeTo`. The amount is dust, but the split never matches the configured percentage exactly.

```
uint onePercentOfLiquidity = liquidity / 100;
uint OnoutLiquidity = onePercentOfLiquidity.mul(OnoutFeePercent);
uint protocolLiquidity = liquidity.sub(OnoutLiquidity);
```

**Recommendation**

Multiply before dividing. The redundant `require` that follows can then be dropped, since `protocolLiquidity + OnoutLiquidity` equals `liquidity` by construction.

```
uint OnoutLiquidity = liquidity * OnoutFeePercent / 100;
uint protocolLiquidity = liquidity - OnoutLiquidity;
```

## SWC-102 | Factory and Router compiled with outdated Solidity 0.8.0

Factory, Pair and RouterV2 were compiled with `v0.8.0+commit.c7dfd78e`, and WrappedOZO with `v0.8.12`. Both releases carry known compiler bugs fixed in later versions, such as `KeccakCaching` and ABI re-encoding issues. None of the listed bugs is reachable from this code, but an old compiler also misses later optimizer and safety improvements.

**Recommendation**

For any redeployment, pin a recent compiler and set `evmVersion` explicitly to what Ozone Chain supports, since compilers from `0.8.20` default to `shanghai` and emit `PUSH0`.

```
pragma solidity 0.8.28;

// hardhat.config.js
solidity: { version: '0.8.28', settings: { evmVersion: 'paris', optimizer: { enabled: true
} } }
```

## SWC-103 | Floating pragma across all contracts

The sources use `^0.8.0`, `^0.8.12` and, in `TransferHelper.sol`, `>=0.6.0`. The deployed bytecode is fixed, but anyone rebuilding or redeploying from this source can pick up a different compiler than the one audited, and the verified source does not state which version the audit covered.

**Recommendation**

Lock every file to the exact compiler version used for the deployment, so a rebuild reproduces the audited bytecode.

```
pragma solidity 0.8.0; // Factory, Pair, RouterV2 and libraries
pragma solidity 0.8.12; // WrappedOZO
```

## ROUTER-ABI | Router quote helpers break the standard Uniswap V2 interface

`getAmountOut()` and `getAmountIn()` take an extra `_factory` argument, so their selectors differ from the standard `getAmountOut(uint,uint,uint)` that aggregators, wallets and bots call; those calls hit no function and revert. All four quote helpers are also non-`view`, and the factory parameter lets a caller quote against any contract, which can mislead integrators.

```
function getAmountOut(address _factory, uint amountIn, uint reserveIn, uint reserveOut)
    public
    virtual
    override
    returns (uint amountOut)
{
    return MainLibrary.getAmountOut(_factory, amountIn, reserveIn, reserveOut);
}
```

**Recommendation**

Expose the standard signatures as `view` functions that use the router's own immutable `factory`, and keep the fee-aware variant under a different name if the front end needs it.

```
function getAmountOut(uint amountIn, uint reserveIn, uint reserveOut)
    public view virtual returns (uint amountOut)
{
    return MainLibrary.getAmountOut(factory, amountIn, reserveIn, reserveOut);
}
```

## FACTORY-CALL-GAS | Every swap and liquidity event makes repeated factory calls

`swap()` calls the factory for `MAX_TOTAL_FEE_PERCENT()`, `totalFee()`, `totalSwaps()` and then `increaseNumberOfSwaps()`, which writes a global counter, and `_mintFee()` makes up to eight more external calls. Each swap pays several thousand extra gas for a statistic nothing on chain uses, and a multi-hop route pays it per hop.

```
uint maxPercent = IFactory(factory).MAX_TOTAL_FEE_PERCENT();
uint totalFee = IFactory(factory).totalFee();
...
if (IFactory(factory).totalSwaps() < type(uint).max)
IFactory(factory).increaseNumberOfSwaps(token0, token1);
```

**Recommendation**

Count swaps off chain from the Swap event instead of a storage write, and read the fee settings once per call, for example through a single `feeConfig()` view returning all values.

```
// Factory.sol
function feeConfig() external view returns (uint total, uint protocol, uint onout, bool
all) {
    return (totalFee, protocolFee, OnoutFeePercent, allFeeToProtocol);
}

// Pair.sol: drop increaseNumberOfSwaps(); index Swap events off chain
```

# AUDIT COMMENTS | Lobsterswap DEX

Smart Contract audit comment for a non-technical perspective

- The admin (feeToSetter, a single EOA) can change the swap fee at any time, up to 99.9% of each trade, with no timelock
- Live configuration sends 100% of swap fees to the protocol feeTo address (protocolFee 10000 and allFeeToProtocol enabled), so liquidity providers currently earn no trading fees
- The admin can redirect the protocol fee recipient (feeTo) and the Onout fee recipient and share (up to 100% of the protocol cut)
- Admin roles transfer in a single step with no zero-address check
- No mint function exists outside liquidity provision: LP tokens are only created against deposited reserves or as the protocol fee share
- No pause, blacklist, whitelist or max transaction limit exists in the Factory, Pair, Router or Wrapped OZO contracts
- The admin cannot withdraw pool reserves or user funds; liquidity can only be removed by LP token holders
- Pair creation is permissionless and every pair is protected by a reentrancy lock
- Wrapped OZO is fully backed 1:1 by OZO held in the contract and has no owner or admin functions
- Contracts are not upgradeable proxies



# CONTRACTWOLF

Blockchain Security - Smart Contract Audits